

MindCo

Guía rápida: instalar, levantar y publicar

App v1.0.0 · junio 2026

Aviso importante sobre las cuentas (léelo primero).

Hoy los servicios que hacen funcionar la app —el contenido (**Strapi**), los usuarios y el login (**Supabase**) y el acceso con Google (**OAuth**)— están dados de alta a **nombre personal del desarrollador** que construyó esta versión. Esto es completamente normal en una etapa inicial.

Cuando el proyecto pase a un nuevo desarrollador o a las **cuentas oficiales de la empresa**, estos servicios (y la base de datos) deberán **crearse de nuevo desde cero** y reconfigurarse bajo las cuentas propias de quien lleve el proyecto. **No es un error ni un pendiente del código**: es el paso natural de traspaso para que la empresa sea dueña de su propia información, sus accesos y sus costos.

 **Plazo a tener muy presente**: estos servidores y servicios se mantendrán activos **únicamente por mes y medio** a partir de la entrega. Pasado ese tiempo pueden apagarse, por lo que conviene **planear el traspaso a las cuentas propias dentro de ese periodo** para que la app siga funcionando sin interrupciones.

1 Lo que vas a necesitar (ya existe, solo ten el acceso a la mano)

Servicio	¿Para qué?	Dato del proyecto
GitHub	Guarda el código.	Rama <code>main</code>
Expo	Prueba y compila la app.	<code>@isra48dev/mindco</code>
Apple Developer	Subir a TestFlight / App Store.	Bundle <code>com.mindco.mvp</code>
Strapi	Panel del contenido (clases, eventos).	<code>useful-vacation-fcb17b63ff.strapiapp.com</code>
Supabase	Usuarios y login (incluye Google).	Proyecto MindCoApp

Programas a instalar una sola vez: **Node.js** (versión "LTS", de `nodejs.org`) y **Git** (de `git-scm.com`; en Mac suele venir incluido). Todos los comandos de esta guía se escriben en la **Terminal** (Mac) o **PowerShell** (Windows): una línea, Enter, esperar.

Antes de empezar: todo el código terminado vive en la rama `main` del repositorio. No tienes que buscar en otras ramas. Si te trabas en un paso, marca cuál es y pásalo a tu apoyo técnico: sabrá exactamente dónde retomar.

2 Descargar el proyecto y armarlo

En la Terminal, ve a la carpeta donde lo quieras guardar (ej. el Escritorio) y escribe esto, línea por línea:

```
git clone <URL-del-repo-de-GitHub>
cd MINDCO_MVP_2026_Master
npm install
```

- `git clone` baja una copia del proyecto (pide la URL exacta a quien administra el GitHub).
- `cd ...` entra a la carpeta.
- `npm install` descarga todo lo que la app necesita. Tarda unos minutos y muestra mucho texto: es normal.

Si `npm install` termina **sin la palabra "error" en rojo**, vas bien.

3 Las "llaves secretas" (archivo `.env`) — con la cuenta administradora

La app necesita unas llaves para conectarse a su contenido (Strapi) y a sus usuarios (Supabase). Por seguridad **no viajan en el repositorio**: se escriben en un archivo local llamado `.env` que tú creas en tu computadora. Ya hay una plantilla: copia `.env.example` y renómbrala a `.env`:

```
cp .env.example .env
```

Ese archivo `.env.example` incluido en el repositorio **ya trae todos los campos con su explicación**, así que te sirve de checklist. Ábrelo con cualquier editor de texto y rellena los valores de **las cuentas que administrarán la app** (ver más abajo cómo obtener cada uno):

Llave	Qué es y de dónde sale
<code>CONTENT_SOURCE</code>	<code>strapi</code> para usar datos reales del CMS, o <code>mock</code> para datos de ejemplo (sin conectar nada).
<code>STRAPI_URL</code>	La dirección de tu proyecto Strapi (sin la "/" final). La obtienes al crear tu Strapi.
<code>STRAPI_API_TOKEN</code>	Llave de solo lectura que se genera dentro de tu Strapi (Settings → API Tokens → "Read-only").
<code>SUPABASE_URL</code>	La dirección de tu proyecto Supabase (Project Settings → API → Project URL).
<code>SUPABASE_ANON_KEY</code>	La llave pública de tu Supabase (la <code>anon</code> / <code>publishable</code> , empieza con <code>sb_publishable_</code>).

🔑 **Cada instancia se crea con la cuenta que administrará la app.** Como se explicó en el aviso de arriba, el Strapi y el Supabase actuales son temporales. El siguiente paso es, **desde la cuenta con la que se administrará la app**, dar de alta una **instancia propia de Strapi, de Supabase y de los demás servicios** para obtener sus direcciones y llaves, y pegarlas en este `.env`. La buena noticia: el repositorio ya incluye las guías para hacerlo.

Cómo sacar cada secreto (todo está documentado en el repo)

Servicio	Qué hacer	Guía dentro del proyecto
Strapi (contenido)	Crea un proyecto nuevo en Strapi Cloud, arma el modelo de datos (clases, eventos, home) y genera un API Token read-only . Copia la URL y el token al <code>.env</code> .	<code>STRAPI_SETUP_PROMPT.md</code> (trae el modelo de datos exacto y los pasos)
Supabase + Google (usuarios y login)	Crea un proyecto Supabase nuevo, copia su URL y su llave pública, y conecta el acceso con Google (OAuth). Copia ambos valores al <code>.env</code> .	<code>docs/supabase-auth-setup.md</code> (paso a paso de Auth + Google)
Cambiar a datos reales	Una vez con tu Strapi listo, pon <code>CONTENT_SOURCE=strapi</code> y reinicia con <code>npx expo start --clear</code> .	<code>README-integration.md</code>

Verificación: estos tres archivos (`.env.example`, `STRAPI_SETUP_PROMPT.md` y `docs/supabase-auth-setup.md`) **ya vienen incluidos en el repositorio**, con la estructura y el orden exacto de los pasos. Tu apoyo técnico solo tiene que seguirlos.

⚠️ **Nunca** subas el archivo `.env` al repositorio ni lo mandes por chat: contiene secretos. El proyecto ya está configurado para ignorarlo automáticamente.

4 Probar la app en tu teléfono (lo más rápido)

Sin compilar nada, para ver la app funcionando:

1. Instala **Expo Go** (gratis) en tu teléfono, desde App Store o Google Play.
2. En la Terminal, dentro de la carpeta, escribe `npm run start`.
3. Escanea el **código QR** que aparece (iPhone: con la app **Cámara**; Android: desde **Expo Go**).

El teléfono y la computadora deben estar en la **misma red Wi-Fi**.

5 Generar la versión final de la app (build de release)

Cuando quieras crear la **versión final e instalable** de la app, se usa **EAS** (el servicio de Expo que compila en la nube). Esto **no implica todavía publicar en ninguna tienda**: solo produce el archivo de la app ya armado. La configuración ya está lista; inicia sesión en Expo y compila:

```
npx eas-cli login
npx eas-cli build --platform ios --profile production
```

- EAS sube el proyecto a la nube y lo compila (10–30 min). Al terminar te da un **enlace con el archivo final** de la app.
- Para Android, el mismo comando cambiando **ios** por **android**.

⚠ **Hazlo en una Terminal real**, sentado frente a la computadora: la primera vez la plataforma te pedirá **iniciar sesión con la cuenta de desarrollador** (Apple / Google) para crear el certificado. Eso no se puede automatizar.

Lo bueno: los valores de producción (Strapi, Supabase, etc.) ya están dentro del perfil **production** del proyecto, así que el build de la nube ya sabe qué usar.

6 Opcional: publicar en las tiendas (decisión posterior)

Una vez que tienes la versión final, **publicarla o no es una decisión aparte**, que se toma cuando ya se quiera dar el paso a las tiendas de cada dispositivo:

- **iPhone:** se sube a **TestFlight** (para que un grupo de personas la pruebe antes) y de ahí a la **App Store**, desde **App Store Connect**.
- **Android:** se sube a la **Google Play Console** (con pruebas internas opcionales antes de publicar).

El envío a la tienda también se hace con EAS, cuando se decida dar ese paso:

```
npx eas-cli submit --platform ios
```

En resumen: el **paso 5** ya te deja la app lista para instalarse. **TestFlight y las tiendas son un paso siguiente y opcional**, que se decide más adelante según hasta dónde se quiera llevar la app.

7 3 pendientes antes de publicar a usuarios reales

1. Falta el ícono real (1024×1024). Hoy usa el ícono por defecto de Expo. Solo hay que entregar la imagen oficial de MindCo y colocarla.

2. Ajustar el login con Google para la versión final. Funciona, pero tiene un ajuste temporal de pruebas; antes del build hay que cambiar el "Site URL" de Supabase a **mindco://auth-callback**. Es rápido pero necesario.

3. Cuenta de Apple definitiva. El identificador **com.mindco.mvp** es de pruebas; al definir la cuenta oficial de la empresa se transfiere la app.

Seguridad: antes de un build de producción confirma que esté **CONTENT_SOURCE=strapi**, **DEV_SKIP_AUTH=false** y **CONTENT_PREVIEW_MODE=false**. Nunca dejes el modo que salta el login en una versión para usuarios.

Mini-glosario. **Repositorio:** el "Drive" del código (GitHub). · **main**: la versión oficial y terminada. · **Terminal:** la ventana donde escribes comandos. · **.env**: archivo local con las llaves secretas. · **Build:** la app ya armada y lista para instalar. · **EAS:** el servicio de Expo que arma el build en la nube. · **TestFlight:** herramienta de Apple para repartir la app a testers. · **CMS / Strapi:** panel donde se sube el contenido. · **Supabase:** servicio que guarda los usuarios y el login. Toda la configuración técnica detallada está dentro del proyecto: **CLAUDE.md**, **README.md**, **README-integration.md** y la carpeta **docs/**.